

第十一章：并发控制——多用户同时改数据怎么办

根本矛盾

数据库的"承诺"是：多个用户能同时使用，每个用户都看到一致的、正确的数据。

但现实是：多用户并发改同一份数据，天然会"打架"——可能丢修改、读脏数据、读两次结果不一样。

第十一章要解决的核心问题：怎么让多用户并发访问看起来"像串行执行"——既不互相干扰，又充分利用并发？

关键认知：并发控制有 4 大技术（封锁、时间戳、乐观控制法、多版本并发控制）。本章只讲封锁——这是最经典、应用最广的一种。考试如果问"并发控制技术有几种"——答 4 种全名；本章只深入讲封锁。

一、并发控制的必要性——3 种数据不一致

不并发的时候，一切太平；一并发就出问题。源材料列了 3 种典型"不一致"——用具体数字场景秒懂：

1.1 丢失修改 (Lost Update)

场景：T1 和 T2 都读 A=10；T1 改 A=11；T2 改 A=12；T1 提交 A=11；T2 提交 A=12。T1 的修改丢了。

T1: 读 A=10 → 算 A+1 → 写 A=11
T2: 读 A=10 → 算 A+2 → 写 A=12

结果：A=12，但 T1 也"以为"自己提交了——T1 的修改被 T2 覆盖。

1.2 不可重复读 (Non-repeatable Read)

场景：T1 读 A=10；T2 改 A=20 并提交；T1 再读 A=20。T1 两次读结果不一样。

T1: 读 A=10 → → 读 A=20 （同事务内两次读，结果不同）
T2: 改 A=20 → 提交

结果：T1 在一个事务里读到了不同的值——可能基于"错的旧值"做决策。

1.3 读脏数据 (Dirty Read)

场景：T1 改 A=10 并写盘；T2 读 A=10；T1 被撤销（回滚），A 恢复 5。T2 读了 T1 没提交的"假数据"。

T1: 改 A=10 → 写盘 → 撤销(回滚) → A 恢复 5
T2: 读 A=10

结果：T2 读到了 A=10，但数据库里其实是 A=5——T2 拿到的数据是"脏"的（T1 撤销后这个值根本不存在）。

3 种问题的对比：

不一致	谁搞的事	严重度	隐蔽性
丢失修改	写-写冲突	必出问题	一眼可见
不可重复读	读-写-再读	隐藏很深	反复读才能发现



读脏数据	写-读	隐藏极深	等撤销后才知道
------	-----	------	---------

恍然大悟点：这 3 种不一致都需要"加锁"来解决——但加锁的严格程度不同（下一节讲 3 级封锁协议）。

二、封锁（X 锁、S 锁）——解决"互斥访问"

封锁 = 事务在操作数据前先给数据加锁，"锁住"之后其他事务要么等、要么只能做有限操作。

2.1 X 锁（排他锁 / 写锁）

源材料第 366-367 行定义：

若事务 T 对数据对象 A 加上 X 锁，则只允许事务 T 读取和修改 A。

>

事务 T 释放 A 上的锁之前：其他事务不能读取和修改 A，*不能再对 A 加任何类型的锁*（重点）。

2.2 S 锁（共享锁 / 读锁）

源材料第 371-372 行定义：

若事务 T 对数据对象 A 加上 S 锁，则事务 T 可以读取 A，但不能修改 A。

>

事务 T 释放 A 上的 S 锁之前：其他事务可以读取 A，可以对 A 加 S 锁，但不能修改 A，也不能对 A 加 X 锁。

2.3 封锁类型相容矩阵

T2 → T1 ↓	X 锁	S 锁	-（无锁）
X 锁	N	N	Y
S 锁	N	Y	Y
-（无锁）	Y	Y	Y

- Y = Yes，相容（允许）
- N = No，不相容（拒绝）

矩阵的 2 条铁律：

1. 只要 T1 加了 X 锁，T2 干啥都不行（X+X=N, X+S=N, X+-=N）—— 因为 X 锁的本质是"独占"
2. 只要 T1 加了 S 锁，T2 还能加 S 锁（S+S=Y）—— 因为 S 锁的本质是"共享读"

类比：

- X 锁 = 一次只让 1 人通过的地铁闸机（独占）
- S 锁 = 多个检票口同时检票（共享读）

2.4 加锁语义速记

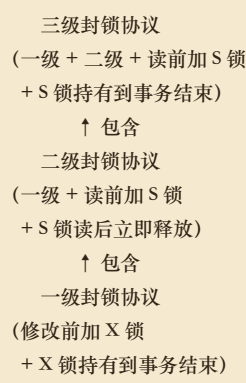
加锁类型	加锁事务能干啥	其他事务能加什么锁
X 锁	读 + 修改	不能加任何锁（X/S/- 都不行）
S 锁	读	能加 S 锁，不能加 X 锁



三、3 级封锁协议 —— "什么程度的一致性"

源材料第 393-431 行明文：3 级封锁协议逐级加强——

3.1 协议的金字塔结构



任何遵守 3 级协议的事务自动遵守 1 级和 2 级协议——这是协议"逐级加强"的核心。

3.2 3 级协议详细对比（重新整理，源材料 401-431 行 OCR 严重混乱）

协议	X 锁加锁时机	X 锁释放时机	S 锁加锁时机	S 锁释放时机	防什么
一级	修改前	事务结束	(不要求)	(不要求)	不丢失修改
二级	修改前	事务结束	读取前	读完即可	一级 + 不读脏数据
三级	修改前	事务结束	读取前	事务结束	一级 + 二级 + 可重复读

为什么 3 级越来越严？

- 一级只管"修改"的串行化（防写-写冲突）
- 二级加了"读取"的串行化（防读-写冲突）
- 三级把"读取锁"也拉到事务结束（防再读时被改）

3.3 一致性保证与 3 种数据不一致的对应

一致性	1 级	2 级	3 级
不丢失修改	防	防	防
不读脏数据	不防	防	防
可重复读	不防	不防	防

速记口诀：

1 级防"丢"，2 级加"脏"，3 级加"重"（"重"= 重复读）

四、活锁 vs 死锁 —— 怎么"卡住"

加锁也可能出"等不到"或"互相等"的问题。

4.1 活锁 —— "等不到"

源材料第 434-436 行例子：

T1 锁 R, T2 等待; T3 也请求 R, T1 释放后T3 获得; T4 又请求 R, T3 释放后T4 获得; T2 永远等不到。

本质: T2 一直在请求, 但每次都被新来的 T3、T4 抢走——T2 被"插队"了。

比喻: 食堂打饭——你排了半天, 每次刚轮到你, 新来的人直接挤到你前面。你永远打不到饭。

解决: 先来先服务。T2 排第一就 T2 先, 不让新来的人插队。

4.2 死锁——"互相等"

源材料第 439-441 行例子:

T1 锁了 R1, 又请求 R2; T2 锁了 R2, 又请求 R1。T1 等 T2 释放 R2, T2 等 T1 释放 R1——永远等不到。

比喻: 单行道的两个司机——A 占了左车道想往右走, B 占了右车道想往左走, 两人互不相让。

活锁 vs 死锁核心区别:

维度	活锁	死锁
谁在等	T2 一个人等	T1 和 T2 互相等
能不能继续	系统还在动 (其他事务正常)	系统完全卡住 (涉及事务都不动)
比喻	排队被插队	单行道两车互不相让
解决	先来先服务	预防 / 诊断+解除

4.3 死锁的解决——预防 vs 诊断与解除

方案	做法	优缺点
预防 (事前避免)	一次封锁法: 一次性把要用的数据全锁上 顺序封锁法: 所有事务按相同顺序加锁	简单但可能降低并发 (一次锁太多数据)
诊断与解除 (事后处理)	超时法: 等待超时判定死锁 等待图法: 有向图检测回路 解除: 撤销代价最小的事务	复杂但不损失并发

等待图法:

- 节点 = 运行中的事务
- 边 = 事务等待关系
- 有回路 = 死锁 (和拓扑排序判断环是同一思路)

五、可串行化调度 (含冲突可串行化) —— 怎么算"正确"

光加锁还不够——怎么判定一个并发调度结果是"正确"的?

5.1 可串行化调度的定义

源材料第 452-454 行:

多个事务并发执行的结果 = 这些事务按某种次序串行执行的结果——才认为这个并发调度是正确的。

举例: T1 和 T2 并发执行:

调度方式	结果	正确?
------	----	-----



T1 串行 T2	A=11, B=20	(基准)
T2 串行 T1	A=12, B=10	(基准)
并发执行 (结果同 T1 串行 T2)	A=11, B=20	正确 可串行化
并发执行 (结果同 T2 串行 T1)	A=12, B=10	正确 可串行化
并发执行 (结果不同于任何串行)	A=12, B=20	错误 不可串行化 (错)

5.2 冲突操作

源材料第 456 行: "冲突操作: 指不同的事务对同一数据的读写操作 (读和写) 和写写操作 (写和写) "。

按标准定义, 冲突操作 = 不可交换的操作:

操作对	是否冲突	原因
读-读	不冲突	谁先读都一样
读-写	冲突	顺序影响结果
写-读	冲突	顺序影响结果
写-写	冲突	后写覆盖前写

源材料用"读写 (读和写)"简写, 实际包含"读-写"和"写-读"两种。

5.3 冲突可串行化

冲突可串行化 = 在调度中, 冲突操作的相对次序与某个串行调度相同。

核心关系:

冲突可串行化 属于 可串行化 (冲突可串行化是可串行化的子集)

即:

- 冲突可串行化一定是可串行化 [正确]
- 可串行化不一定是冲突可串行化 [正确] (存在"非冲突可串行化"的可串行化调度)

六、两段锁协议 (2PL) —— "充分条件"

源材料第 460-463 行明文。

两段锁协议 = 任何事务必须分两阶段对数据加锁和解锁:

阶段	干什么	比喻
扩展阶段 (Growing)	申请加锁——可加锁, 不能释放	装修: 只买材料 (进门)
收缩阶段 (Shrinking)	释放锁——可释放, 不能再申请	装修: 只装东西 (出门)

关键约束:

释放一个封锁之后, 事务不再申请和获得任何其他封锁 —— 一旦进入"收缩", 就不能回头"扩展"。

6.1 2PL 的核心性质

若并发执行的所有事务均遵守两段锁协议, 则对这些事务的任何并发调度策略都是可串行化的。

这是充分条件 (不是必要):

- 遵守 2PL → 一定可串行化 [正确]

• 可串行化 → 不一定遵守 2PL [正确] (2PL 是管得严的协议, 有些可串行化调度并不遵守 2PL)
 比喻: 2PL 像"先全部加完锁再开始释放"的纪律——只要大家都遵守这个纪律, 调度一定可串行化。但不遵守 2PL 也不一定错, 只是没"保险"。

七、跨章呼应：全课程视角

第十一章是"并发控制"——实现 ACID 中的"隔离性", 与全课程都有连接:

章节	关联点
第 1 章 绪论	DBMS 统一管理和控制的 4 大任务之一 = 并发控制
第 4 章 安全性	并发访问时, 每个用户仍要符合授权 (GRANT/REVOKE)
第 5 章 完整性	完整性检查 vs 封锁 (都是"互斥"机制) ——5 章检查数据的"对", 本章保证数据的"序"
第 7 章 数据库设计	维护阶段"性能监督" = 监控并发性能
第 8 章 数据库编程	嵌入式 SQL 程序 = 一个或多个事务 = 本章的并发对象
第 10 章 恢复技术	撤销 (UNDO) = 死锁解除 / 检查点恢复 / 事务回滚——都靠"撤销"操作
第 11 章 本章	ACID 的隔离性 = 并发控制的目标

OCR 错误处理

源材料第 8-9 页有少量 OCR 错误, 订正如下:

源材料原文 (错误)	应该是
第 446 行 "解决活锁的方法" (前面讲死锁)	解决死锁的方法
第 456 行 "对用一数据"	对同一数据
第 401-431 行 3 级协议表格行列错乱	已重新组织为 3×5 表格

易混淆对比总结

表 1: X 锁 vs S 锁

维度	X 锁 (写锁)	S 锁 (读锁)
别名	排他锁	共享锁
加锁事务能干啥	读 + 修改	只能读
其他事务能加 X 锁?	不行	不行
其他事务能加 S 锁?	不行	行
其他事务能读?	不行	行
比喻	一次只让 1 人通过的闸机	多个检票口同时检票

表 2: 3 种数据不一致



不一致	谁搞的事	严重度	隐蔽性	防它的最低协议
丢失修改	写-写冲突	必出	一眼可见	1 级
读脏数据	写-读-撤销	隐藏	等撤销才发现	2 级
不可重复读	读-写-再读	隐藏	反复读才发现	3 级

表 3：3 级封锁协议

协议	X 锁加锁	X 锁释放	S 锁加锁	S 锁释放	防什么
1 级	修改前	事务结束	—	—	不丢失修改
2 级	修改前	事务结束	读取前	读完即可	+ 不读脏
3 级	修改前	事务结束	读取前	事务结束	+ 可重复读

表 4：活锁 vs 死锁

维度	活锁	死锁
谁在等	一个人等	互相等
系统是否卡住	还在动	完全卡住
比喻	排队被插队	单行道两车互不相让
解决	先来先服务	预防 / 诊断+解除

表 5：可串行化 vs 冲突可串行化

维度	可串行化	冲突可串行化
定义	并发结果 = 某串行结果	冲突操作次序与某串行一致
关系	概念更广	是可串行化的子集
判定难度	难	相对容易
关系	冲突可串行化 是 可串行化 的子集	—

表 6：封锁类型相容矩阵

$T2 \rightarrow T1 \downarrow$	X 锁	S 锁	-
X 锁	N	N	Y
S 锁	N	Y	Y
-	Y	Y	Y

逻辑主线



多用户并发访问 → 数据可能不一致



[为什么需要并发控制] 3 种数据不一致

- ├— 丢失修改（写-写冲突）
- ├— 不可重复读（读-写-再读）
- └— 读脏数据（写-读-撤销）



[怎么解决] 4 大技术：封锁、时间戳、乐观控制法、多版本并发控制

本章只讲封锁 ↓



[封锁的 2 种基本锁]

- ├— X 锁（写锁）：独占，其他事务不能加任何锁
- ├— S 锁（读锁）：共享读，其他事务可加 S 锁但不能加 X 锁
- └— 相容矩阵（3×3）



[加锁的纪律] 3 级封锁协议

- ├— 1 级：X 锁持有到事务结束（防丢失修改）
- ├— 2 级：1 级 + S 锁读后释放（防读脏）
- └— 3 级：1 级 + S 锁持有到事务结束（防不可重复读）



[加锁的问题] 活锁 vs 死锁

- ├— 活锁：等不到 → 先来先服务
- ├— 死锁：互相等
- ├— 预防：一次封锁法 / 顺序封锁法
- └— 诊断与解除：超时法 / 等待图法 / 撤销代价最小的事务



[什么算“正确”] 可串行化调度

- ├— 可串行化 = 并发结果 = 某串行结果
- ├— 冲突操作 = 读-写、写-读、写-写（不可交换）
- └— 冲突可串行化 是 可串行化 的子集



[充分条件] 两段锁协议（2PL）

- ├— 扩展阶段：申请加锁
- ├— 收缩阶段：释放锁（释放后不能申请）
- └— 遵守 2PL → 一定可串行化（**充分非必要**）



跨章呼应：第 1 章 DBMS 4 大任务 / 第 4 章授权 / 第 5 章完整性 / 第 7 章维护 / 第 8 章事务 / 第 10 章 UNDO

整个第十一章就一件事：让多用户并发访问“看起来像串行执行”——用封锁解决互斥，用封锁协议保证一致性，用可串行化判定正确性，用 2PL 提供充分条件。

整个数据库系统（第一/四/五/七/八/十/十一章）也讲一件事：DBMS 用 4 大机制（安全性/完整性/恢复/并发控制）统一管理数据，让共享既高效又正确。

